Engineering Design & Tech Stack

ECITY — Mobile Shop Management Web App

Version 1.5 · 2026

TypeScript stack, architecture, self-hosted on one server

Companion documents: PRD, Module Breakdown, Engineering Design, Deployment Guide, Database Guide

1. Purpose and Constraints

This document specifies **how** to build the product described in the PRD, for a small team — realistically one developer — whose strongest language is **TypeScript / JavaScript**, with **shadcn/ui** for the interface, and with a hard eye on **monthly running cost**.

Constraints that shaped every choice below:

Constraint	Consequence
TypeScript/JavaScript is the familiar language	One language across frontend, backend, scripts, tests and migrations. No Java, Go, Python or PHP in the critical path
One developer, ~30 modules of scope	Prefer a single deployable application over microservices; prefer boring, well-documented tools over clever ones
Money, stock and IMEI records must be exactly right	A relational database with real transactions and constraints. Not negotiable
Reporting and analytics are half the product	SQL-first data layer. The reports are joins and aggregates, and a document database would make them painful
Monthly cost must stay small and predictable	**One rented Linux server running everything in Docker**; no managed-service subscriptions, no per-seat SaaS in the critical path. Target under ₹1,000/month
shadcn/ui requested for styling and components	React + Tailwind CSS, which fixes the frontend framework family

2. Recommended Stack

Layer	Choice	Why this one
Language	**TypeScript 5.x** (strict)	Your language; one type system from the database row to the React prop. Strict mode is what makes it worth having
Framework	**Next.js 16 (App Router)**	Frontend and backend in one deployable TypeScript project. Server Components keep heavy report queries on the server; Route Handlers give you a normal REST API for the counter screens
UI components	**shadcn/ui** + **Radix UI**	As requested. Components are copied into your repo, so you own and can adapt them — right for a dense back-office app with tables, dialogs, comboboxes and command palettes
Styling	**Tailwind CSS v4**	Required by shadcn/ui; keeps a 40-screen app visually consistent without a growing stylesheet
Forms & validation	**React Hook Form** + **Zod**	One Zod schema validates the browser form and the server handler, and infers the TypeScript type. Removes a whole class of bugs in a form-heavy product
Server state	**TanStack Query**	Anything Postgres owns: caching, background refetch, optimistic updates, retry — matters on the billing screen and on a shaky shop connection. Introduced in M4; before that, Server Components read the service layer directly and nothing is fetched
Client state	**Zustand** (from M4)	The bill being built at the counter — cart lines, discounts, attached customer, split payments, trade-in. Four or five sibling components read and write it, which is past what props or context handle well. ~1 KB, no provider, a store is just a hook. *Not* Redux: its strengths (time-travel devtools, middleware, team conventions) do not pay for their boilerplate here
Tables	**TanStack Table** (headless, styled with shadcn)	Inventory, sales and report grids need sorting, filtering, pagination and column visibility. Do not hand-roll this

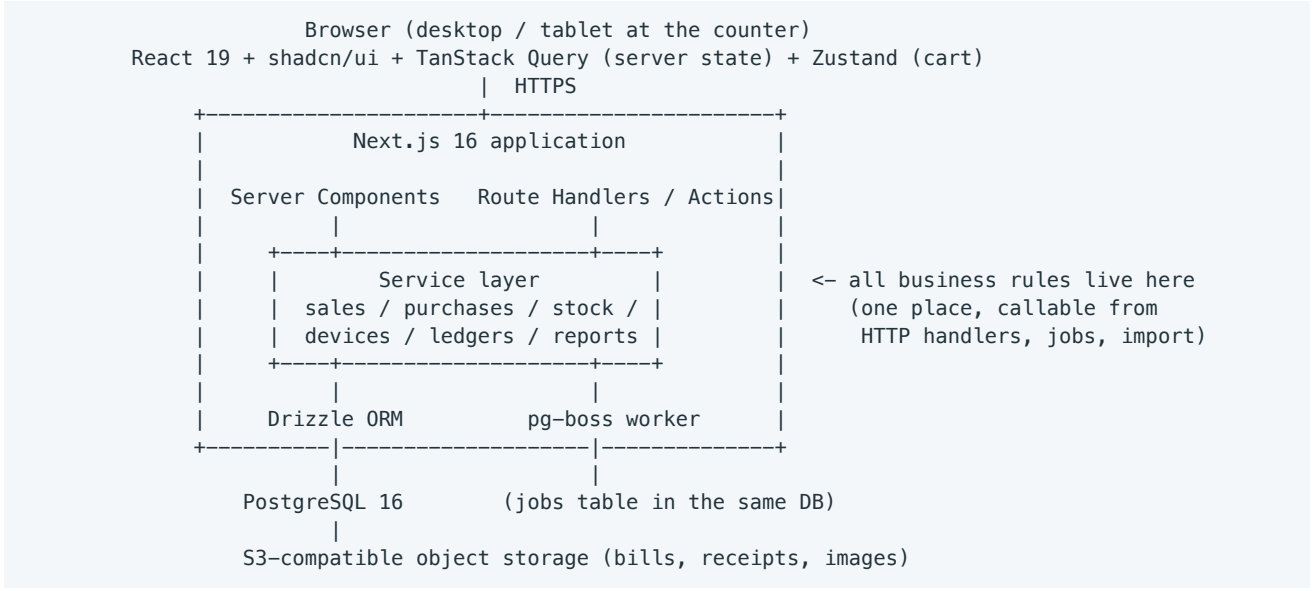
Built as: TanStack Table has **not** been adopted yet. Through M4 the grids need paging and filtering but not client-side sorting or column visibility, and both are served by plain server components: filters and the page number live in the URL, and one shared `<Pagination>` renders the control. Adopt TanStack Table at the first screen that needs client-side sorting or column toggles — M10's dashboards are the likely trigger. | Charts | **Recharts** | React-native API, adequate for the dashboards and analytics in PRD §6.15 | | Database | **PostgreSQL 16+** | Transactions, foreign keys, check constraints, partial and composite indexes, window functions for analytics, full-text search for global search, `numeric`/`bigint` for money. Everything this product needs is in the box | | ORM / query layer | **Drizzle ORM** | TypeScript-first, generates SQL you can read, migrations are plain SQL files, and dropping to raw SQL for a report is trivial. *Alternative:* Prisma, if you prefer its DX — both are fine; Drizzle wins on report-heavy work and cold-start cost | | Auth | **Own session layer** — random token in an httpOnly cookie, SHA-256 hash stored in Postgres | Sessions in your own database, revocable, with sliding idle expiry. Auth.js v5 was the original choice, but its Credentials provider only supports JWT sessions — database sessions are not available with it, and PRD FR-1.1 requires revocable server-side sessions. Roughly 150 lines (`src/server/auth/session.ts`), no vendor, no library to fight | | Authorisation | Own permission layer (`requirePermission(user, perm, branchId)`) | Roles are editable sets of permissions plus a branch scope; no library models this well. Optionally reinforced with Postgres Row-Level Security later | | File storage | **Cloudflare R2** (S3-compatible) | Bill photos, receipts, product images. Free tier covers 10 GB with zero egress fees. Keep it behind a small internal interface so any S3-compatible provider can replace it | | Background jobs | **pg-boss** (job queue inside Postgres) | Alerts, nightly rollups, import processing. Runs as a second container beside the app — it needs a long-lived process, which is one more reason not to deploy serverless. No Redis, no extra service, no extra bill | | PDF / print | Browser print + CSS `@page` for A4 and 80 mm; **@react-pdf/renderer** for downloadable

invoice PDFs | Avoids running headless Chrome, which is the expensive way to make a PDF | | Excel / CSV export | **exceljs**, streaming CSV | Handles the export requirements in FR-25.4 and FR-33.3 without loading everything into memory | | Testing | **Vitest** (unit), **Playwright** (end-to-end) | One test runner for TS; Playwright to prove the six core business flows in PRD §7 keep working | | Error tracking | **Sentry** | Free tier is sufficient at this scale | | Hosting | **One AWS Lightsail instance in Mumbai**, everything in Docker | App, PostgreSQL, worker and Caddy (TLS) as four containers on one machine. 1 GB for staging, 2 GB for production; ≈ ₹1,200–1,400/month for both. See §6 and the Deployment Guide | | CI/CD | **GitHub Actions** | Typecheck, lint, test, build an image, push to GHCR, SSH to the server and restart. The server never compiles anything |

2.1 Deliberate rejections

Rejected	Reason
MongoDB / Firebase / any document store	The product is inherently relational — sales, items, devices, ledgers, branches — and needs multi-row transactions and aggregate reporting. This choice would be paid for daily
Serverless hosting (Vercel, Lambda) for this app	Three costs with no matching benefit here: cold starts of 300 ms–1 s on the billing screen, a connection storm against Postgres that needs a pooler to survive, and no long-lived process for the pg-boss worker. A single always-on Node process has none of these problems and is cheaper
Managed database as a service (Supabase, Neon, RDS)	₹1,300–2,200/month for something a Docker container does for ₹0 on a box you are already renting. What you actually buy is managed backups — worth it at scale, not at one shop. §6 and the Deployment Guide replace it with three backup layers
Separate NestJS/Express backend repo	Doubles the deployment, auth plumbing and type-sharing work for a solo developer with no benefit at this scale. Keep one app; the service layer inside it is the seam if you ever split
Microservices	There is no scaling or team problem here that they solve, and several correctness problems they create
Floating-point money (`float`, `double`, JS `number` for amounts)	Rounding errors in a system whose whole purpose is reconciling cash. Store integer paise in `bigint`
A BaaS for auth (Clerk, Auth0) at $25+/mo	Cost with no benefit: your roles and branch scoping are custom regardless
Redux / Redux Toolkit	Four times the code for the same cart. Its real advantages are for large teams and time-travel debugging, neither of which applies. Zustand covers the one screen that needs shared client state
A global client store for server data	Inventory and sales live in Postgres. Mirroring them into a store means writing cache invalidation by hand, badly. Server Components and TanStack Query already solve it
Prisma Accelerate / paid data proxies	Not needed at this connection count

3. Architecture



One rule keeps this maintainable: HTTP handlers, background jobs and the CSV importer all call the *same* service functions. `sellDevice()` is written once, so a device sold through the UI, through an import or through a correction job produces the same stock movement, the same ledger rows and the same `device_event`.

3.1 Repository layout

```
/app                Next.js routes (route groups per module)
  /(auth)           login, reset
  /(app)/[branch]   dashboard, billing, inventory, purchases, ...
  /api              route handlers
/components         shadcn/ui primitives + app components
/server
  /db               drizzle schema, migrations, seed
  /services         sales, purchases, stock, devices, ledger, reports, search
    /external-import adapters + mapping for the other billing system's Excel
                    (see the note below - only the adapter knows the layout)
  /auth             session, permissions, branch scope
  /jobs             pg-boss workers: alerts, rollups, imports, backups
/lib                zod schemas, money helpers, formatting, constants
/tests              vitest unit, playwright e2e
```

4. Data Layer Decisions

These thirteen decisions are the difference between a system that reconciles and one that does not.

4.1 Money is `bigint` paise. Every amount column is integer minor units. A `Money` helper type handles arithmetic, and formatting to ₹ happens only at the display edge. No `float`, no `number` arithmetic on amounts in JavaScript.

4.2 Balances are derived, never stored as mutable counters. Customer outstanding, supplier outstanding, cash-drawer cash and account balances are all sums over append-only ledger tables. Cache them in a summary column if a report demands it, but the ledger is the truth and a nightly job must be able to prove the cached value equals the recomputed one.

4.3 `device_event` is append-only and is the source of the IMEI history.

```
create table device_event (
  id          bigserial primary key,
  device_id   bigint  not null references device_unit(id),
```

```
  seq          int     not null,              -- ordering within a device
  event_type   text    not null,              -- purchased | received | transferred_out |
                                              -- transferred_in | reserved | sold | returned |
                                              -- inspected | reclassified | repaired |
                                              -- damaged | lost | adjusted | voided
  occurred_at  timestamptz not null,
  branch_id    bigint references branch(id),
  from_branch_id bigint references branch(id),
  to_branch_id   bigint references branch(id),
  ref_type     text,                       -- purchase|sale|transfer|return|adjustment
  ref_id       bigint,
  actor_id     bigint references app_user(id),
  payload      jsonb  not null default '{}',
  unique (device_id, seq)
);
create index on device_event (device_id, seq);
```

No `UPDATE` and no `DELETE` are permitted on this table — revoke those privileges from the application role. The device history page in PRD §6.21 is one indexed read of this table joined out to its referenced documents.

4.4 A device's IMEIs are a separate table, not `imei_1` / `imei_2` columns. A handset can carry more than one IMEI, and some carry more than two. Storing them as numbered columns means a schema migration, a data backfill and a rewrite of every form, query, importer and report the day a third one appears. Storing them as rows costs one join today and nothing later.

```
create table device_identifier (
  id          bigserial primary key,
  device_id   bigint   not null references device_unit(id) on delete restrict,
  imei        text     not null,
  slot        smallint not null default 1,       -- 1 = first SIM slot
  is_primary  boolean  not null default false,
  created_at  timestamptz not null default now(),
  constraint imei_digits check (imei ~ '^[0-9]{14,17}$'),
  unique (imei),                                 -- unique across the whole business
  unique (device_id, slot)
);

-- exactly one primary identifier per device
create unique index device_identifier_one_primary
  on device_identifier (device_id) where is_primary;

-- partial-IMEI search
create index device_identifier_imei_trgm
  on device_identifier using gin (imei gin_trgm_ops);
```

`device_unit.primary_imei` is kept as a cached copy, maintained by a trigger on `device_identifier`, so that lists, invoices and exports do not join for the common case. It is a display convenience and never the source of truth — all matching, validation and search run against `device_identifier`.

The UI shows one IMEI field in v1, and that is a setting, not a limitation. `business_setting.imei_slots` (default `1`) controls only how many IMEI inputs the purchase and product forms render. It has no effect on validation, storage, search, import or export: the service API always takes and returns `imeis: string[]`, and a device that arrives from an import with three identifiers is stored, searched and reported with all three while the setting still reads 1. Raising it to 2 or 3 later is a row in a settings table — no migration, no backfill, no release.

```
// service API — the shape never changes when the setting does
createDeviceUnit({ productId, branchId, mainType, isNewCut, imeis: string[], ... })
findDeviceByImei(imei)          // matches ANY identifier of any device
```

4.5 The GLOBAL / NEW CUT rule is enforced by the database, not only by code.

```
alter table device_unit add constraint new_cut_only_global
  check (is_new_cut = false or main_type = 'GLOBAL');
```

Application validation can be bypassed by an import, a script or a bug. A check constraint cannot.

4.6 Concurrency on device sale. Two tills must never sell the same IMEI. Guard it with a conditional update inside the sale transaction rather than an application-level read-then-write:

```
update device_unit
   set status = 'SOLD'
 where id = $1 and status = 'IN_STOCK' and current_branch_id = $2
returning id;   -- zero rows returned  =>  abort the sale with a clear message
```

4.7 Indexes that must exist from day one. `device_identifier(imei)` unique plus its trigram index; `device_unit(current_branch_id, status, main_type)`; `device_event(device_id, seq)`; `sale(branch_id, created_at)`; `sale_item(device_id)`; `customer(phone)`, `supplier(phone)`; and GIN trigram indexes on customer, supplier and product names for global search. Add a `pg_trgm` extension for partial IMEI and name matching.

4.8 Reporting strategy. Live SQL for anything within the current month. For year-range analytics, a nightly `pg-boss` job writes per-branch, per-day rollups (sales, cost, profit, units, payment mix, stock value) and the analytics pages read rollups for closed days plus live queries for today. This keeps PRD §9.1's five-second target reachable without a warehouse.

4.9 A document records the regime it was issued under, not the one in force today. Nothing archives the invoice PDF — `/api/sales/[id]/pdf` renders from the rows on every request — so every reprint is a fresh render and the row has to carry everything the document needs to look like itself. `sale.prices_included_tax` was the first of these; `sale.gst_enabled` (FR-2.6) is the second. The general rule: **when a setting decides how a past document reads, snapshot it onto the document.** Read it live only where the value describes the shop now, never where it describes a document then. Watch for the tempting shortcut of inferring the setting from the figures — zero tax does not mean "no GST regime", because exempt and zero-rated goods sold under GST are also zero.

4.10 A signed statement is stamped, not recomputed. A daily closing (PRD FR-13, OQ-5) records what a person counted and signed for. Its expected, counted and difference figures are stored on the row and never rewritten; the reports recompute from `cash_movement` and are *meant* to diverge once a correction lands, which is surfaced rather than reconciled away. This is the same principle as §4.9 applied to a number rather than a format: read live where the value describes the shop now, stamp where it describes what someone attested to then. Everything else about money stays derived — expected cash is opening plus the sum of the movements, and every account balance is its opening plus its transactions (§4.2). The rule is not "stamp figures"; it is "stamp attestations".

4.11 Search re-applies every rule the rest of the application enforces. Global search (M9, FR-30) touches every table at once, which makes it the single easiest place to leak something every other screen is careful about. So each branch of it re-applies the caller's branch scope *and* checks the permission for the kind of record it is about to return — there is deliberately no blanket permission on the endpoint, because one would either lock out staff who legitimately search or hand them rows they cannot open. Where two rules appear to conflict, both are satisfied rather than one dropped: a customer record is business-wide (FR-6.7) but a branch-limited user must not reach another branch's customer (FR-30.7), so visibility follows the trade — reachable if they have bought at a visible branch, or have not bought anywhere yet.

4.12 "Today" is the shop's day, never the browser's or the server's. `new Date().toISOString().slice(0, 10)` is UTC, which in India is a *different day* between midnight and 05:30 — the till is open, the drawer is on today's business date, and the browser thinks it is yesterday. Found when a test run crossed midnight IST: the expense form defaulted to the previous day, and capped its own date picker below the day the shop was standing in, so nobody could record that evening's expense at all. `src/lib/date.ts` holds one definition of the shop's calendar day and both sides use it — the server's `businessDateFor` and every form default — so a form and the drawer it posts into cannot drift apart. Anything user-facing that means "today at the shop" comes from there.

4.13 Analytics read one query layer, and a limited user gets nothing rather than everything. Nine areas (M10, FR-16 – FR-24) share one shape — range, branch set, comparison — so two screens cannot disagree

about the same month. The branch resolver returns a *concrete list* whenever the caller is limited, never "no filter": an empty filter meaning *unfiltered* is the leak that matters, so a branch-limited user asking for a branch they cannot see gets zero, not the whole business. Margins sit behind their own permission (`analytics.view_profit`), the same line `inventory.view_cost` already drew. No rollup tables: live queries answer a twelve-month range well inside PRD §9.1's five seconds, and a nightly rollup would add a refresh to keep honest and a staleness window to explain.

4.14 Data arriving in bulk uses the same door as data typed in. An importer that writes to tables directly is how a shop ends up with stock the ledger has never heard of and devices with no event history — records the rest of the system does not recognise, discovered months later by a report that will not reconcile. So every row goes through `createDevice`, `createParty`, `createProduct`, `increaseStock` and the ledger services, exactly as the forms do; the importer's job is parsing and mapping, never persistence. Two consequences follow, and both are deliberate. Bulk import is slower than a `COPY`, which does not matter for a job run once at go-live. And an opening balance is a *movement* like any other (§4.2): stock arrives as an `OPENING` stock movement, cash as an `OPENING` cash movement, a debt as an `OPENING` ledger entry — never a balance column, which would be the one number in the system nobody could prove. The other half of the rule is refusal. Structural problems reject the file whole, because importing "the rows that happened to parse" is silent partial data; row problems are named by their line *as the spreadsheet shows it* and the rest still goes in; and a name the file mentions but the shop does not have stops that row rather than creating the party, because a due against a name nobody recognises is worse than a missing row.

4.15 A spec describes a handset, not a catalogue entry — and it is captured where the goods arrive. "iPhone 17" is the product; 256GB green at 87% battery is a *unit*. Encoding specs into the product multiplies it out — four storages by six colours is twenty-four catalogue entries for one model, none of which exist until the stock walks in, so the catalogue would be created at the goods-inwards desk by whoever was standing there, in whatever spelling they used, and every report that groups by product would quietly fragment. So `device_unit` holds `ram`, `storage`, `colour`, `variant` and `battery_health_percent`, and the product stays one row.

That leaves the entry problem, which is the real one: a spec nobody has time to type is a spec nobody records. It is captured on the **purchase line**, which already implies one combination — a line has a single unit cost, and a 256GB does not cost what a 128GB costs — and the line stamps every unit it creates. Any unit may override it for the odd piece in a batch. The same values are columns in the device importer, so a file and a typed-in purchase produce identical handsets. And the stamp is one-way: correcting the line later does not rewrite handsets already created, by the rule in §4.9 — a record keeps what was true when it was made, and a handset is corrected on its own.

4.16 A list states its own size, and sorts on the server. A list that silently stops at twenty-five reads as *that is everything*, which is how stock goes missing — so every list screen carries a control saying the range and the total, even when there is only one page. Sorting is a **link**, not client state: it lands in the URL beside the filters and the page number, so a sorted view survives a reload and can be pasted to someone. That is not only tidiness. Client-side sorting over a paged table sorts *the page you can see* and presents the result as though it were the list — an answer that is confidently wrong. Sorting on the server means the rows you get are the right rows, and a new sort returns to page 1, because staying on page 7 of a re-ordered list shows a stranger's rows.

Where the paging happens depends on what the list *is*. Users, roles and branches are bounded by the business — one row per employee, per role, per branch — and every picker in the app already reads those functions whole, so they are fetched whole, sorted and cut for display; paging them in SQL would fork the query to buy nothing. Stock, sales and low stock grow with trade, so they page and sort in the database. Roles are the deliberate exception to the control itself: it is a master-detail editor, and paging the picker would mean turning a page to reach the role you came to edit, so it states its count and stops there. And a sort key arriving from the URL is user input like any other — it is checked against the columns the screen offers before it can reach an `ORDER BY`.

The control itself has to survive the layout. These tables become cards below `md`, so column headings alone would have meant *sorting does not exist on a phone* — on an app whose smallest tested screen is 320px and whose owner checks stock from the shop floor. The cards get a scrollable strip of the same links instead, which is how it

was caught: the sorting tests passed on desktop and tablet and failed on both phone widths, because there was nothing there to click.

4.17 An alert is a condition, and its audience is decided when it is read. The tempting shape for notifications is a row per person — it makes "my unread list" a single indexed query. It is also wrong here, because it fixes the audience at write time: a user assigned to another branch, a new hire, a role that gains `closing.view` next week, and the shop is left with alerts addressed to people who have moved on, or with none at all for the person now responsible. So one row per *condition*, carrying a branch, and read through the same `branchScope` and permission checks as every other query (§4.11, §4.13) — a branch user sees their branches plus what is business-wide, an owner sees all of it grouped, and a kind whose screen the reader cannot open is filtered out before the query returns. Read state is the one genuinely personal thing, so it is its own table: one manager clearing the bell must not hide a till shortage from the owner.

The second half is not shouting. The evaluator runs on a schedule, so every notification carries a **dedupe key** identifying the condition (`LOW_STOCK:branch:product`), and the same key is not raised while it is still open. Resolving matters as much as raising: a key that never closed would leave a dealt-with alert on the screen *and* block the next genuine occurrence forever, so each pass reports which conditions still hold and everything else is closed. That is also why turning a rule off clears what it had already raised — an alert nobody can explain the origin of is worse than no alert.

4.18 A guard that cannot fail is not a guard. Two of M14's checks were written, run, and found to be checking nothing. The restore drill counted append-only triggers by a name pattern that matched none of them — the triggers were fine, the check was decoration. The destructive-path audit looked for `export const DELETE` and flagged an endpoint that *voids*, while a real delete elsewhere in a service would have passed unnoticed. Both were rewritten to test the behaviour rather than its shape: the drill now attempts a forbidden write against the restored copy and expects to be refused, and the audit reads the service layer for actual deletes of money and stock. The general rule this leaves: a safety check should be made to fail on purpose once, before it is trusted — and if it cannot be made to fail, it is not testing anything.

4.19 Performance is a property of the data, not of the code. Every §9.1 target passed on a developer's database of three thousand devices. On the PRD's own sizing assumption — 250k devices, a million events, 900k sales — the dashboard missed its two-second target by 60%, and the cause was a query that read *every completed sale in the shop's history* to print one total: it discarded settled bills in JavaScript, after the database had already shipped them. Cost scaled with everything ever sold rather than with what was still owed. The fix was in two parts, and the second is the more useful lesson: pushing the filter into SQL made it proportional to debts (3.3s → 1.9s), but the real win was noticing the dashboard never wanted the aged rows at all and giving it an aggregate that never leaves the database (1.9s → 60ms). Optimising a query is worth less than not asking it. This is why `scripts/perf-seed.sql` exists and why the perf suite refuses to run without it: a timing taken on a small database is not a weak measurement, it is a misleading one.

5. Cross-Cutting Implementation Notes

Authorisation. Every route handler and server action begins with `requirePermission(user, permission, branchId)`. Write an automated test that enumerates every endpoint and asserts a 403 for a user lacking the permission or the branch — this is the single most valuable test suite in the project, because the failure mode (Branch A staff reading Branch B money) is invisible until it is embarrassing.

Branch scope. Resolve the active branch (or the permitted branch set for consolidated views) once per request and thread it through the service layer as an explicit argument. Never read it from a global.

Idempotency. The billing screen generates a UUID per bill attempt and sends it as an idempotency key. The server stores it uniquely; a retry after a dropped connection returns the original sale instead of creating a second

one. Do the same for payments and transfers.

Audit. A thin wrapper over the write path records actor, entity, action and changed fields into `audit_log` automatically, so a new feature is audited without the developer remembering.

Barcode / IMEI scanning. Scanners behave as keyboards. The billing search input listens for fast character bursts terminated by Enter and treats them as a scan; no drivers, no native app. A scan resolves through `device_identifier`, so scanning the label's second IMEI finds the device just as well as the first.

Printing. Two print stylesheets — A4 invoice and 80 mm thermal — driven by CSS `@page` and `@media print`. Downloadable PDFs use `@react-pdf/renderer` on the server. Avoid headless Chrome; it is the single most expensive thing you could add to your hosting bill.

Uploads. The browser requests a short-lived signed upload URL and posts directly to object storage; the server stores only the key. Downloads are served through short-lived signed URLs, never a public bucket.

Where state lives. Three tiers, and putting something in the wrong one is the most common way a React codebase rots:

Tier	Tool	Examples
Server state — Postgres owns it	Server Components reading services directly; **TanStack Query** for client-side reads	Inventory, sales, customers, dues, the audit log
Client state — shared across components, exists only in the browser	**Zustand**	The in-progress bill: cart lines, discounts, split payments, attached trade-in
Local state — one component owns it	`useState`	A dialog's open/closed, a form error, a draft filter

Never copy server data into a client store. It goes stale immediately and you end up hand-writing cache invalidation that TanStack Query already does. M0–M3 need no store at all: every piece of client state is local to one component.

The billing store also carries the bill's **idempotency key** and is persisted to `localStorage`, which is what satisfies PRD §9.3 — a dropped connection or an accidental refresh must not lose a half-built bill, and re-submitting must not create a second one.

Integrating the other billing system. The shop keeps a second system for NEW items and exports Excel from it daily (PRD §6.22). The column layout is unknown at design time, so isolate it: `external-import/types.ts` defines a canonical row shape that the rest of the pipeline is written against, `adapters/*.adapter.ts` is the only code that knows the real column names, and `mapping/legacy.mapping.json` holds the column-name mapping so a changed header is a config edit rather than a release. Staging and preview before commit, idempotency on `(source, external invoice no, external line id)` plus a file hash, and apply through the *same* service functions as manual entry — never straight into tables. The one rule that makes this safe rather than merely tidy: a device whose `sales_channel` is `EXTERNAL` cannot be sold on the ECITY billing screen at all, so the two systems can never invoice the same IMEI.

Environments. `local` (Docker Compose on your laptop) → `staging` → `production`. Staging is either a second small Lightsail instance or a second Docker stack on the same box with its own database and port; either way it costs little and it is what stops a bad migration reaching real sales data. Migrations run in CI against staging before they are allowed near production.

6. Hosting and Monthly Running Cost

Everything runs on **one rented Linux server**. The app, the database, the background worker and the TLS proxy are four Docker containers on the same machine, talking to each other over Docker's internal network. There is no managed database, no serverless platform and no per-seat subscription anywhere in the critical path.

Prices are indicative for **September 2026**, converted at approximately **₹88 = $1**. Confirm on the provider's own pricing page before committing — regional prices differ from headline list prices.

6.1 The recommended setup

> **Revised September 2026.** This section previously recommended Hetzner CX22 in Singapore at ≈₹400–600. That advice was wrong on both counts — the CX line is EU-only, so "CX22 in Singapore" described a machine that does not exist, and Hetzner raised cloud prices 144% in June 2026. See Deployment Guide §1, which carries the full comparison.

AWS Lightsail, Mumbai. One instance, everything in Docker.

Item	Choice	₹ / month
Server (staging)	**Lightsail Mumbai**, 1 vCPU / 1 GB / 40 GB	~₹500 + 18% GST
Server (production)	**Lightsail Mumbai**, 2 vCPU / 2 GB / 60 GB	~₹1,060 + 18% GST
Automated snapshots	Lightsail snapshots (~20% of instance)	~₹100–200
Object storage + off-site backups	Cloudflare R2, free tier (10 GB, zero egress)	₹0
Transactional email	Resend free tier (3,000/month)	₹0
Error tracking	Sentry Developer, free	₹0
Domain	`.in`, ~₹800/year	~₹70
Total, both environments		**≈ ₹1,200–1,400**

Why Mumbai over a cheaper European box. 20–30 ms from Kerala against roughly 150 ms, and the shop's sales records stay in India — one fewer thing to explain to a CA. Lightsail bills a flat monthly figure including a generous transfer allowance rather than metering every component the way EC2 does.

What to know before choosing it. Lightsail has **no in-place resize**: to grow you snapshot, build a larger instance from it and move the static IP across (Deployment Guide §8b), and you cannot go back down afterwards. Attach a static IP on day one or the new instance comes up on a different address and DNS breaks. DigitalOcean Bangalore and Vultr Mumbai resize in place at comparable prices, and are the alternatives if that matters more than staying inside AWS.

Sizing. Start staging at 1 GB — the server never compiles anything, it pulls a pre-built image, so Postgres, the app and Caddy fit comfortably. Provision **production at 2 GB**: Node holds 200–400 MB under load, Postgres wants its buffers, and the analytics queries are the memory-hungry part. Since Lightsail will not let you shrink later, the safe order is small for staging, right-sized for production.

Step-by-step setup, hardening, deployment pipeline, backup scripts and the operational runbook are in the companion **Deployment Guide**.

6.2 The alternatives, if Lightsail's resize rule bites

Provider	Region	1 vCPU / 2 GB, ₹ / month	Resizes in place
Vultr	Mumbai	≈ ₹880 + 18% GST	Yes
DigitalOcean	Bangalore	≈ ₹1,060 + 18% GST	Yes
Linode / Akamai	Mumbai	≈ ₹1,060 + 18% GST	Yes

Same architecture, same containers, same guide — every provider gives you an Ubuntu machine and an IP address. Nothing in this design is Lightsail-specific except the upgrade procedure.

6.3 Managed hosting, for later

Kept here as the upgrade path, not the starting point. Vercel Pro (~₹1,760) plus a managed Postgres such as Supabase Pro or Neon (~₹1,670–2,200) comes to **≈ ₹3,500–4,000/month**, roughly six times the VPS. What that money buys is managed backups, automatic scaling and no server administration.

That trade is not worth making at one shop with ten branches, for three reasons: the load is a few percent of one small server (§7), serverless adds cold starts to the billing screen and a connection-pool problem to solve, and pg-boss needs a long-lived worker that serverless does not provide. Revisit it when the ops burden genuinely outweighs ₹3,000/month — not before.

6.4 What you are accepting by self-hosting

	Consequence	Mitigation
Backups are yours	Nobody else is protecting the data	Three layers and a monthly restore drill — Deployment Guide §7
Single point of failure	Server dies, app and database go together	Rebuild + restore in 30–60 minutes, practised in advance
OS and patching are yours	Security updates, certificate checks, disk space	`unattended-upgrades`, Caddy auto-renews, ~2–4 hours a month

The honest cost of self-hosting is not risk, it is **your time**: about half a day to set up and two to four hours a month thereafter. That buys back roughly ₹35,000 a year.

6.5 Portability rules that keep the exit route open

Self-hosting is only the cheap option if leaving stays cheap. Four rules:

1. **Plain PostgreSQL only.** No provider-specific database features. The same schema and migrations must run on your VPS, on Supabase, on Neon or on RDS.
2. **Your own session table**, never a hosting provider's auth service. This is the rule that most often traps people on a platform.
3. **S3-compatible storage behind a small internal interface**, so Cloudflare R2 ↔ any other bucket is a config change.
4. **The app runs in Docker locally.** If it runs in a container on your laptop, it runs anywhere.

Moving to managed hosting later is then `pg_dump` → `pg_restore` → change `DATABASE_URL`. One evening.

6.6 What actually grows the bill

Driver	Effect	Mitigation
Uploaded bill photos and product images	The fastest-growing line, and the one that could push R2 past its free 10 GB	Compress and resize on upload; cap file size; archive attachments older than three years
Database size (device events, ledgers)	~2 M events over five years is only a few GB — modest	Partition `device_event` by year if it ever matters
Backup retention	Compressed dumps defeat deduplication and multiply storage	`pg_dump -Fc -Z 0` and let restic compress; 7 daily / 4 weekly / 6 monthly
Long-range analytics queries	CPU, and the temptation to rent a bigger box	The nightly rollups in §4.8 are the fix; buy indexes before hardware
Headless-Chrome PDF generation	Would need a bigger server on its own	Keep to `@react-pdf/renderer` and browser printing
Adding WhatsApp / SMS notifications	Per-message cost, typically ₹0.20–₹0.90 each	Decide via PRD OQ-6; keep messaging opt-in per rule

At the PRD §9.1 sizing (10 branches, 50 users, ~500 bills a day) none of this needs an upgrade. The first real increase arrives only if attachments outgrow R2's free tier.

---

7. Performance and Scale

Running the backend inside Next.js on one small server is not a constraint at this size. The arithmetic, taking the PRD sizing and doubling it:

	Estimate
Requests per day (billing, search, dashboards, browsing)	~100,000
Average across a 12-hour trading day	**~2.5 req/s**
Realistic peak, every branch busy	**~30 req/s**
What one always-on Node process serves, with a DB query each	**200–500 req/s**
Database size after five years	a few GB
IMEI lookup on a unique index over 2 M rows	sub-millisecond

Peak load is roughly **5–10% of one CX22**. The shop would need to grow twenty to fifty times before the architecture, rather than a tuning detail, became the question.

7.1 What actually degrades first

1. **Unbounded report queries.** "All branches, all products, three years" scanning everything is the only thing here that genuinely hurts — and it hurts identically in any language or framework. The nightly rollups in §4.8 exist for this.
2. **N+1 queries.** A device list that fetches identifiers per row turns one query into two hundred. Catch it in review; log slow queries in development.
3. **Memory.** A report that materialises a large result set gets the process killed by the OOM killer. Stream exports; paginate everything; never `SELECT *` a whole table into JavaScript.
4. **Disk filling up.** The most common way a small server actually stops working. Old Docker images are the usual culprit — `docker image prune` runs on every deploy.

Note what is *not* on this list: the framework, the language, and the fact that the backend and the frontend share a project.

7.2 Headroom and the upgrade path

Trigger	Response	Downtime
CPU or RAM consistently above 70%	Move to the next Lightsail plan — snapshot, rebuild larger, move the static IP (Deployment Guide §8b)	~20 minutes, after closing
Database outgrows the disk	The same upgrade: Lightsail disk grows with the plan	~20 minutes
Sustained load above ~100 req/s	Split the database onto its own server	An evening
Second team, separate deploy cadence	Extract `/server/services` as its own API	Weeks — and only then

This is why every business rule lives in `/server/services` and never inside a route handler: the split stays cheap without being paid for now.

8. Testing Strategy

Level	Tool	What it covers
Unit	Vitest	Money arithmetic, tax computation, aging buckets, expected-cash calculation, GLOBAL/NEW CUT validation
Service / integration	Vitest + a real Postgres in Docker	Every service function against a real database, including transaction rollback and the concurrent-sale race
Authorisation	Vitest, table-driven	Every endpoint × every role × in-scope and out-of-scope branch
End-to-end	Playwright	The six core business flows in PRD §7, run on every push
Data integrity	Scheduled job	Recompute every ledger balance and compare with what the UI shows; alert on any drift

The concurrency test and the authorisation matrix are the two suites that repay their cost repeatedly. Write them early.

9. Risks and Mitigations

Risk	Impact	Mitigation
ER and ACT are still undefined (PRD OQ-1)	Wrong stock model, discovered after M4 when it is expensive	Answer before M2 starts. Model `main_type` as an enum + a per-type attribute table so a definition change is a data change, not a migration of every table
The device/event schema is rewritten mid-build	Weeks lost; history gaps that cannot be reconstructed	Design M0–M2 schema fully up front; treat `device_event` as an API with a version, and never mutate it
Reports built ad hoc per screen	Same number computed three ways, three different answers	One analytics query layer with a shared date/branch/grouping contract (M10)
A device turns out to need a second or third IMEI after launch	Would be a migration plus a rewrite of every form, query and report if identifiers were columns	Already handled: identifiers are rows from the first migration (§4.4) and the UI limit is the `imei_slots` setting
Backups are never tested	The one failure that turns an outage into a dead business	Monthly restore drill, in M13's "Done when" and in the calendar — Deployment Guide §7.2
Server disk quietly fills up	Postgres stops accepting writes and the shop cannot bill	`docker image prune` on every deploy; weekly disk check with an alert at 80%
The other system's Excel format changes, or was never as regular as assumed	The daily feed breaks and stock silently drifts	One adapter plus a JSON column mapping; fixture tests built from real exports; the reconciliation report surfaces drift the next morning
The shop stops uploading the daily file	Stock, cash and closing all go wrong quietly	Closing is gated on the feed; a standing dashboard alert while it is missing
Solo-developer bus factor	The project stops	Everything in git, migrations in the repo, `README` with the runbook, no manual production changes ever. A second SSH key stored offline
Scope creep from "just one more report"	The 29-week plan becomes 50	The module list is the contract; new requests are queued for v1.1
Offline requirement arrives late (PRD OQ-7)	The sales module is substantially rebuilt	Decide before M4. If it is genuinely needed, budget an extra 3–4 weeks and design the sale as a client-generated, idempotent document from the outset

10. Getting Started Checklist

1. `npx create-next-app@latest --typescript --tailwind --app`
2. `npx shadcn@latest init`, then add: `button card input table dialog dropdown-menu form select command sheet tabs toast badge popover calendar`
3. `npm i drizzle-orm postgres drizzle-kit zod react-hook-form @hookform/resolvers @tanstack/react-table recharts pg-boss date-fns` (add `@tanstack/react-query` and `zustand` at M4, when the billing screen first needs them — not before)
4. `npm i -D vitest @playwright/test @types/pg eslint prettier`
5. Docker Compose with Postgres 16 for local development

6. GitHub repository, GitHub Actions for typecheck + lint + test + build, deploy on `main`
7. Provision the Lightsail instance and Cloudflare R2 bucket, and wire the environment variables — follow the Deployment Guide
8. Write the M0–M2 schema in full before writing the first screen, `device_identifier` and `imei_slots` included
9. Add the money helper (`bigint` paise) and forbid `number` amounts by lint rule
10. Begin Module M0